# PROJECT REPORT

**PROJECT TITLE: Garbage Waste Management System**

**NAME: GHUFRAN AHMED KHAN**

**CMS: 3510-2024**

**Instructor: Mr. Prem Sagar**

**Course: Data Structures and Algorithms (Lab)**

**Section: BS-SE-3A**

**Dated: 20-Jan-2026**

## Introduction:

This project is based on the concept of a **Garbage waste Management System**, whose primary objective is to efficiently manage and clean garbage from different locations of a city based on priority. In urban environments, waste management is a major challenge, and it becomes important to clean the most critical areas first using limited resources. This system addresses that problem by prioritizing locations with higher garbage levels.

The project demonstrates how **Data Structures** can be applied to solve a real-world problem in an organized, logical, and efficient way. Instead of cleaning locations randomly, the system ensures that locations with the highest garbage fill level are handled first.

## Problem Statement:

In real-life garbage management, multiple locations produce waste at different rates. Cleaning all locations simultaneously is not possible due to limited manpower and resources. Therefore, there is a need for a system that:

- Stores information about multiple locations
- Identifies which location requires immediate cleaning
- Maintains a record of recently cleaned locations
- Works efficiently even as the number of locations increases

This project provides a structured solution to this problem using appropriate data structures.

## System Overview:

The Garbage waste Management System manages different locations, each having a **name** and a **fill level** representing the amount of garbage collected at that location. Based on this fill level, the system decides which location should be cleaned first.

The system follows a simple workflow:

1. Locations are added to the system with their fill levels
2. Locations with higher fill levels are given higher priority
3. The highest priority location is selected for cleaning

4. After cleaning, the record is stored for history tracking

This workflow ensures fairness, efficiency, and better decision-making.

## Data Structures Used:

This project makes use of multiple data structures to achieve its objectives efficiently. Each data structure is selected based on its specific role in the system.

## Array for Location Storage:

An **Array** is used to store all the location records. Each location contains details such as the location name and its garbage fill level. The array acts as the base data storage structure of the system.

The use of an array allows:

- Easy access to location data
- Simple and organized storage
- Efficient indexing and traversal

Since the number of locations is known at runtime, arrays provide a suitable and efficient solution.

## Priority Queue Using Max Heap:

To manage garbage cleaning based on priority, the system uses a **Priority Queue implemented with a Max Heap**. The fill level of a location determines its priority. Locations with higher fill levels are considered more critical and are cleaned first.

A Max Heap is used because:

- The location with the highest fill level always stays at the root
- Removal of the highest priority location is efficient
- Priority-based decision making becomes fast

This ensures that the system always selects the most urgent location for cleaning.

## Heap Operations:

### Heapify Up Operation

Whenever a new location is added or a fill level is updated, the **Heapify Up** operation is performed. This operation moves the newly added element upward in the heap until the Max Heap property is restored.

Heapify Up ensures that:

- The highest fill level location reaches the correct position
- Heap structure remains valid after insertion
- Priority order is maintained

### Heapify Down Operation

When the highest priority location is cleaned and removed from the heap, the **Heapify Down** operation is used. This operation moves the root element downward to restore the heap structure.

Heapify Down ensures that:

- The next highest priority location becomes the new root
- The Max Heap property is preserved
- The system remains efficient after deletion

## Stack for Cleaning History:

After a location is cleaned, its record is stored in a **Stack**. The stack follows the **Last In First Out (LIFO)** principle, which is ideal for tracking the most recent cleaning activities.

The use of a stack allows:

- Easy access to the most recently cleaned location
- Efficient push and pop operations
- Better tracking and monitoring of cleaning history

This feature is helpful for reporting, analysis, and future decision-making.

## Manual Implementation of Data Structures:

In this project, all data structures such as **Array, Heap, Priority Queue, and Stack** are implemented manually instead of using Java's built-in libraries. This approach improves understanding of internal working mechanisms and strengthens core programming concepts.

Manual implementation helps in:

- Learning how data structures work internally
- Gaining better control over logic and operations
- Enhancing problem-solving and debugging skills

## System Efficiency and Scalability:

The Garbage waste Management System is designed to be efficient and scalable. The use of a Priority Queue ensures fast decision-making, while the stack provides an organized history of cleaned locations.

As the number of locations increases:

- Priority-based operations remain efficient
- The system continues to perform reliably
- Resource utilization stays optimized

This makes the system suitable for real-world scenarios.

## Conclusion:

The Garbage waste Management System successfully demonstrates the practical application of data structures in solving a real-world problem. By combining Arrays, Priority Queue (Max Heap), and Stack, the system efficiently manages garbage cleaning priorities and maintains cleaning history.

This project highlights how proper data structure selection can improve system performance, decision-making, and organization. Overall, the system is well-structured, efficient, and scalable, making it a strong example of how data structures can be applied in real-life garbage management solutions.

## PROJECT CODE:

```java
import java.util.Scanner;

public class GarbageManagementSystem {

    // ===== System Data =====
    Location[] locationList;   // sari locations ki list
    int locationCount;         // kitni locations add ho chuki hain

    PriorityQueue pq;          // Max Heap based priority queue
    CleanHistoryStack history; // Cleaned locations ka record (Stack)

    // ===== Constructor =====
    GarbageManagementSystem(int capacity) {
        locationList = new Location[capacity];   // fixed size array
        pq = new PriorityQueue(capacity);        // heap size
        history = new CleanHistoryStack(capacity); // history stack
        locationCount = 0;
    }
```

```java
Run | Debug
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print(s: "\t Enter number of locations: ");
    int capacity = sc.nextInt();
    sc.nextLine(); // newline consume

    GarbageManagementSystem system = new GarbageManagementSystem(capacity);

    int choice;

    do {
        System.out.println(x: "\n\t====================================");
        System.out.println(x: "\t   GARBAGE MANAGEMENT SYSTEM MENU");
        System.out.println(x: "\t====================================");
        System.out.println(x: "\t 1. Add New Location");
        System.out.println(x: "\t 2. Update Garbage Fill Level");
        System.out.println(x: "\t 3. Show All Locations");
        System.out.println(x: "\t 4. Show Next Location to Clean");
        System.out.println(x: "\t 5. Clean a Location");
        System.out.println(x: "\t 6. Show Cleaning History");
        System.out.println(x: "\t 7. Exit System");
        System.out.print(s: "\t Enter your choice: ");

        choice = sc.nextInt();
        sc.nextLine(); // newline consume
```

```
switch(choice) {

    // ===== Add Location =====
    case 1:
        System.out.print(s: "\n\t Enter location name: ");
        String name = sc.nextLine();

        System.out.print(s: "\t Enter fill level (%): ");
        float fillLevel = sc.nextFloat();

        system.addLocation(name, fillLevel);
        break;

    // ===== Update Fill Level =====
    case 2:
        System.out.print(s: "\n\t Enter location name to update: ");
        String locName = sc.nextLine();

        System.out.print(s: "\t Enter new fill level (%): ");
        float newLevel = sc.nextFloat();

        system.updateFillLevel(locName, newLevel);
        break;
```

```
    case 3:
        system.displayAllLocations();
        break;

    // ===== Next Location =====
    case 4:
        Location next = system.getNextLocationToClean();
        if(next != null)
            System.out.println("\n\t Next location to clean: "
                    + next.name + " (" + next.fillLevel + "%)");
        else
            System.out.println(x: "\n\t No locations available.");
        break;

    // ===== Clean Location =====
    case 5:
        system.cleanLocation();
        break;

    // ===== History =====
    case 6:
        System.out.println(x: "\n\t ===== Cleaning History =====");
        system.displayCleanHistory();
        break;
```

```
            case 7:
                System.out.println(x: "\n\t System exited successfully!");
                break;

            default:
                System.out.println(x: "\n\t Invalid choice! Please try again.");
        }

    } while(choice != 7);

    sc.close(); // Scanner band
}
```

```
// ===== Add Location =====
// Duplicate check + array + heap insert
void addLocation(String name, float fillLevel) {

    if (isDuplicateLocation(name)) {
        System.out.println(x: "\n\t Location already exists!");
        return;
    }

    Location loc = new Location(name, fillLevel);
    locationList[locationCount++] = loc;
    pq.insert(loc);

    System.out.println(x: "\n\t Location added successfully!");
}
```

```
// ===== Update Fill Level =====
void updateFillLevel(String name, float newFillLevel) {

    if (isSystemEmpty()) {
        System.out.println(x: "\n\t System is empty.");
        return;
    }

    for (int i = 0; i < locationCount; i++) {
        if (locationList[i].name.equals(name)) {

            locationList[i].fillLevel = newFillLevel;
            pq.updateFilllevelInQueue(name, newFillLevel);

            System.out.println(x: "\n\t Fill level updated successfully!");
            return;
        }
    }

    System.out.println(x: "\n\t Location not found.");
}
```

```java
// ===== Get Next Location =====
Location getNextLocationToClean() {
    if (pq.isEmpty()) return null;
    return pq.peek();
}

// ===== System Empty Check =====
boolean isSystemEmpty() {
    return locationCount == 0;
}

// ===== All Clean Check =====
boolean allLocationsCleaned() {
    for (int i = 0; i < locationCount; i++) {
        if (locationList[i].fillLevel > 0)
            return false;
    }
    return true;
}
```

```java
// ===== Duplicate Check =====
boolean isDuplicateLocation(String name) {
    for (int i = 0; i < locationCount; i++) {
        if (locationList[i].name.equals(name))
            return true;
    }
    return false;
}
```

```java
// ===== Clean Location =====
void cleanLocation() {

    if (pq.isEmpty()) {
        System.out.println(x: "\n\t No location available to clean.");
        return;
    }

    if (allLocationsCleaned()) {
        System.out.println(x: "\n\t All locations are already clean!");
        return;
    }

    Location cleaned = pq.removeMax(); // heapifyDown yahan hota hai
    cleaned.fillLevel = 0;
    history.push(cleaned);

    System.out.println(x: "\n\t Location cleaned & added to history!");
}

// ===== Display History =====
void displayCleanHistory() {
    if (history.isEmpty()) {
        System.out.println(x: "\t No cleaning history found.");
        return;
    }
    history.display();
}
```

```java
// ===== Display History =====
void displayCleanHistory() {
    if (history.isEmpty()) {
        System.out.println(x: "\t No cleaning history found.");
        return;
    }
    history.display();
}
```

```java
// ===== Display All Locations =====
void displayAllLocations() {

    if (isSystemEmpty()) {
        System.out.println(x: "\n\t No locations available.");
        return;
    }

    System.out.println(x: "\n\t ===== All Locations Status =====");
    for (int i = 0; i < locationCount; i++) {
        System.out.println("\t " + locationList[i].name +
                " --> Fill Level: " + locationList[i].fillLevel + "%");
    }
}
```

```java
// ===== Location Class =====
class Location {
    String name;
    float fillLevel;

    Location(String name, float fillLevel) {
        this.name = name;
        this.fillLevel = fillLevel;
    }
}
```

```java
// ===== PRIORITY QUEUE (MAX HEAP) =====
class PriorityQueue {

    Location[] heap;
    int size;

    PriorityQueue(int capacity) {
        heap = new Location[capacity];
        size = 0;
    }

    boolean isEmpty() {
        return size == 0;
    }
```

```
// ===== Insert with Heapify Up =====
void insert(Location loc) {
    heap[size] = loc;
    heapifyUp(size);
    size++;
}
```

```
// ===== Update Fill Level =====
void updateFilllevelInQueue(String name, float newFillLevel) {

    for (int i = 0; i < size; i++) {
        if (heap[i].name.equals(name)) {

            float oldFill = heap[i].fillLevel;
            heap[i].fillLevel = newFillLevel;

            // agar value barhi → heapifyUp
            if (newFillLevel > oldFill)
                heapifyUp(i);

            // agar value kam hui → heapifyDown
            else if (newFillLevel < oldFill)
                heapifyDown(i);

            return;
        }
    }
}
```

```
Location peek() {
    return heap[0];
}

// ===== Remove Max =====
Location removeMax() {

    Location max = heap[0];
    heap[0] = heap[size - 1];
    size--;

    heapifyDown(index: 0);
    return max;
}
```

```
// ===== Heapify Up =====
void heapifyUp(int index) {

    while (index > 0) {

        int parent = (index - 1) / 2;

        if (heap[index].fillLevel > heap[parent].fillLevel) {
            swap(index, parent);
            index = parent;
        } else break;
    }
}
```

```
// ===== Heapify Down =====
void heapifyDown(int index) {

    int largest = index;
    int left = 2 * index + 1;
    int right = 2 * index + 2;

    if (left < size &&
        heap[left].fillLevel > heap[largest].fillLevel)
        largest = left;

    if (right < size &&
        heap[right].fillLevel > heap[largest].fillLevel)
        largest = right;

    if (largest != index) {
        swap(index, largest);
        heapifyDown(largest);
    }
}
```

```
// ===== Swap =====
void swap(int i, int j) {
    Location temp = heap[i];
    heap[i] = heap[j];
    heap[j] = temp;
}
}
```

```
    // ===== STACK (History) =====
    class CleanHistoryStack {

        Location[] stack;
        int top = -1;

        CleanHistoryStack(int capacity) {
            stack = new Location[capacity];
        }

        void push(Location loc) {
            stack[++top] = loc;
        }

        boolean isEmpty() {
            return top == -1;
        }

        void display() {
            for (int i = 0; i <= top; i++) {
                System.out.println("\t " + stack[i].name);
            }
        }
    }
}
```

## OUTPUTS:

```
Enter number of locations: 2

====================================
   GARBAGE MANAGEMENT SYSTEM MENU
====================================
1. Add New Location
2. Update Garbage Fill Level
3. Show All Locations
4. Show Next Location to Clean
5. Clean a Location
6. Show Cleaning History
7. Exit System
Enter your choice: 1

Enter location name: market
Enter fill level (%): 56.7

Location added successfully!
```

```
====================================
   GARBAGE MANAGEMENT SYSTEM MENU
====================================
1. Add New Location
2. Update Garbage Fill Level
3. Show All Locations
4. Show Next Location to Clean
5. Clean a Location
6. Show Cleaning History
7. Exit System
Enter your choice: 1

Enter location name: plaza
Enter fill level (%): 76.89

Location added successfully!
```

```
====================================
   GARBAGE MANAGEMENT SYSTEM MENU
====================================
 1. Add New Location
 2. Update Garbage Fill Level
 3. Show All Locations
 4. Show Next Location to Clean
 5. Clean a Location
 6. Show Cleaning History
 7. Exit System
 Enter your choice: 2

 Enter location name to update: plaza
 Enter new fill level (%): 90.32

 Fill level updated successfully!
```

```
====================================
   GARBAGE MANAGEMENT SYSTEM MENU
====================================
 1. Add New Location
 2. Update Garbage Fill Level
 3. Show All Locations
 4. Show Next Location to Clean
 5. Clean a Location
 6. Show Cleaning History
 7. Exit System
 Enter your choice: 3

 ===== All Locations Status =====
 market --> Fill Level: 56.7%
 plaza --> Fill Level: 90.32%
```

```
====================================
   GARBAGE MANAGEMENT SYSTEM MENU
====================================
 1. Add New Location
 2. Update Garbage Fill Level
 3. Show All Locations
 4. Show Next Location to Clean
 5. Clean a Location
 6. Show Cleaning History
 7. Exit System
 Enter your choice: 4

 Next location to clean: plaza (90.32%)
```

```
 ====================================
    GARBAGE MANAGEMENT SYSTEM MENU
 ====================================
  1. Add New Location
  2. Update Garbage Fill Level
  3. Show All Locations
  4. Show Next Location to Clean
  5. Clean a Location
  6. Show Cleaning History
  7. Exit System
  Enter your choice: 5

  Location cleaned & added to history!
```

```
 ====================================
    GARBAGE MANAGEMENT SYSTEM MENU
 ====================================
  1. Add New Location
  2. Update Garbage Fill Level
  3. Show All Locations
  4. Show Next Location to Clean
  5. Clean a Location
  6. Show Cleaning History
  7. Exit System
  Enter your choice: 6

  ===== Cleaning History =====
  plaza
```

```
     ====================================
        GARBAGE MANAGEMENT SYSTEM MENU
     ====================================
      1. Add New Location
      2. Update Garbage Fill Level
      3. Show All Locations
      4. Show Next Location to Clean
      5. Clean a Location
      6. Show Cleaning History
      7. Exit System
      Enter your choice: 7

      System exited successfully!
C:\Users\HAJI LAPTOP g55\Desktop\ghufran>
```